



AGENT INFRASTRUCTURE

MCP VS A2A

TWO PROTOCOLS, ONE STACK

Two protocols define how every AI agent works in 2026.
One gives agents tools. The other gives them teammates.
Understanding the difference changes how you build.

MCP

A2A

AGENT CARDS

JSON-RPC

PROTOCOL STACK

AAIF



TWO DIFFERENT PROBLEMS

MCP and A2A are not alternatives — they solve distinct problems at **different layers** of your agent architecture. Treating them as competing standards is the first mistake.

DIMENSION	MCP	A2A
Purpose	Agent → Tools	Agent → Agents
Transport	JSON-RPC 2.0	HTTP + SSE
Remote Reasoning	No	Yes — full AI
Auth Model	Per-server	OAuth 2.0 built-in
Created by	Anthropic (Nov '24)	Google (Apr '25)
Governance	Linux Foundation AALF (Dec '25)	



MCP UNDER THE HOOD

Client-server on JSON-RPC 2.0. Your agent connects to MCP servers that expose **4 capability types**: Resources, Tools, Prompts, and Sampling. The server is the gatekeeper — the model never touches infrastructure directly.

● ● ● mcp_server.py

```
# MCP server — tool exposed to agent
@mcp.tool()
async def query_database(
    sql: str,
    limit: int = 10
) → list[dict]:
    """Execute a read-only query."""
    return await db.execute(sql, limit)

# Transport: stdio (local) or SSE (remote)
# Agent calls this via JSON-RPC 2.0
# 5,800+ public servers · 97M downloads/mo
```



PULKIT TYAGI
Senior Gen-AI Engineer



A2A UNDER THE HOOD

Each agent publishes an **Agent Card** at `/.well-known/agent.json` — a manifest declaring capabilities, inputs, and auth. Tasks move through a formal state machine with real-time SSE updates.



Agent Card — declares capabilities, inputs, auth at discovery time



Task state machine — submitted → working → completed / failed



Server-Sent Events — stream real-time progress for async tasks



OAuth 2.0 built-in — scoped permissions and audit trails included



POST /tasks/send — start work; artifacts returned on completion



PULKIT TYAGI
Senior Gen-AI Engineer



THE PROTOCOL STACK IN 2026

Modern agentic systems run on **three layers** — each handling a different communication boundary. Build from the bottom up.

WebMCP

LAYER 1

Structured web access
— llms.txt, agent-
accessible sitemaps

MCP

LAYER 2

Agent → Tool
integration. Every API,
DB, filesystem your
agent touches

A2A

LAYER 3

Agent → Agent
coordination.
Research ↔ Planning
↔ Execution

Most production systems today: **MCP-only** (single agent + multiple tool servers). Start here. Add A2A when remote reasoning is genuinely required.



WHICH ONE DO YOU NEED?

The question isn't which protocol is better. It's **which layer you're working at**. Using A2A where MCP is sufficient adds latency, complexity, and state machine overhead.

USE MCP WHEN

Connecting to deterministic tools

Interaction is request-response

Server doesn't need to reason

Need broad AI client support

→ **Start every project here**

USE A2A WHEN

Remote system has real AI reasoning

Tasks are async or long-running

Back-and-forth negotiation needed

Agent discovery + capability matching

→ **Add when genuinely needed**



MISTAKES THAT KILL ARCHITECTURES

Most agentic systems fail not because the models are wrong, but because the **protocol layer is misused**.



Treating MCP and A2A as **alternatives** — they are layers, not competitors



Using A2A for **deterministic ops** — overhead with no reasoning benefit



Skipping MCP **server security** — 7,000+ exposed servers with no access controls



Building **custom protocols** when AAIF standards already have 97M downloads



Over-engineering multi-agent systems **before MCP-only is working**



**ARE YOU BUILDING MCP-FIRST,
OR JUMPING STRAIGHT TO
A2A?**



LIKE



COMMENT



FOLLOW